

语义引领的下一代数字化建设体系

——Of AI, By AI, For AI

语义引领的下一代数字化建设体系

——Of AI, By AI, For AI

概述

Of AI

本体论和语义层

知识图谱和GraphRAG

语义层的数据源

语义层的更新

By AI

业务分析与需求梳理

系统设计

开发与测试

监控与运维

For AI

AI Native

存量治理

未来展望

应用的未来形态

组织的未来形态

参考资料

概述

在本文中，我们构想了一套基于人工智能的下一代数字化建设体系。根据信息化、数字化的性质和人工智能的特点，提出，未来的数字化建设应当是语义引领的。语义是统合系统建设和运营全过程的关键，语义层（本体）则是实现这种统合的工具。

我们认为，未来的 IT 系统建设，应当是“AI 所有（Of AI）、AI 所治（By AI）、AI 所享（For AI）”的：

- **Of AI**：即 AI 已经超越了辅助工具的角色；透过语义层，它开始“拥有”整个系统。本体论贯穿IT/数字化活动的全过程；基于本体论构建的语义层承载着对系统的认知，是一切活动的基础。
- **By AI**：即 AI 作为系统构建的工具和方法。从业务分析到需求梳理、从架构计到编码实现、从系统测试到运维监控，AI 在所有的环节都起到辅助甚至替代人的作用。
- **For AI**：即 AI Ready。AI 时代的应用需天然是 AI Ready 的，任何一个拥有其本体知识的智能体都有能力与其进行交互，比如查询数据或执行业务操作。

下面详述这三者。

Of AI

Of AI 是本文的核心。它的职责是对系统进行**语义建模**；它构成系统的“**认知架构**”（或曰“认知基础设施”）。

本体论和语义层

本体论（Ontology）不是新名词，上个世纪90年代就在数据科学领域被提出，后来又成为知识图谱的理论基础，最近则是借着帕兰提尔公司（Palantir）的东风重新回到人们的视野。

1993年，Tom Gruber提出：**本体是概念模型的显式规范**。还有一些类似的说法，比如“**共享概念的形式化和显式说明**”，强调概念的共享性、形式化和明确性，从而突出机器可读性。

这里提一下共享性，即本体中的概念应当是企业全局的，不是隶属于某一个部门、某一个业务或某一套 IT 系统的。我们的语义层建设整体上应当遵循这个观念；但就单个 IT 系统来说，它有自己的领域模型，也有自己的——用DDD术语来说——有界上下文。

共享本体的一个重要职责是消歧，即解决不同部门或系统使用不同的词汇表达同一个概念的问题。

总体而言，本体论包含如下核心要素：

1. **概念/类**：企业中的基本“事物”类型。

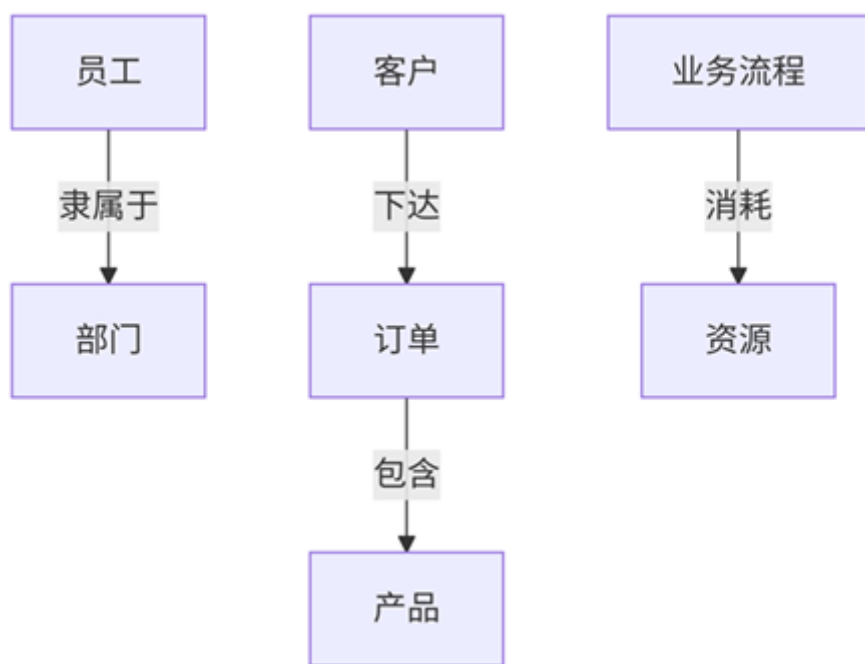
例如：客户、产品、订单、员工、部门、业务流程、项目、资源。

2. **属性**：描述这些概念的特定信息或特征。

例如：客户有客户ID、姓名、地址。订单有订单号、下单日期、金额。

3. **关系**：概念之间的互动和联系方式。这是本体论强大之处。

例如：员工隶属于部门。客户下达订单。订单包含产品。业务流程消耗资源。画成图如下：



4. **规则/公理**：约束企业运作的逻辑规则。它们定义了“业务逻辑”。

例如：“一个订单必须至少包含一个产品”。“只有经理级别的员工才能审批金额超过10万元的合同”。“已完成状态的订单不能再被修改”。

5. **实例**：根据上述本体论定义创建的具体数据。

例如：一个具体的客户（实例：张三，ID：C001），他下达的一个具体订单（实例：订单O2025001）。

基于该理论的发展，业内出现了一批实现，如 OWL（Web Ontology Language，当时web技术正在兴起）、语义网络、本体工程等。

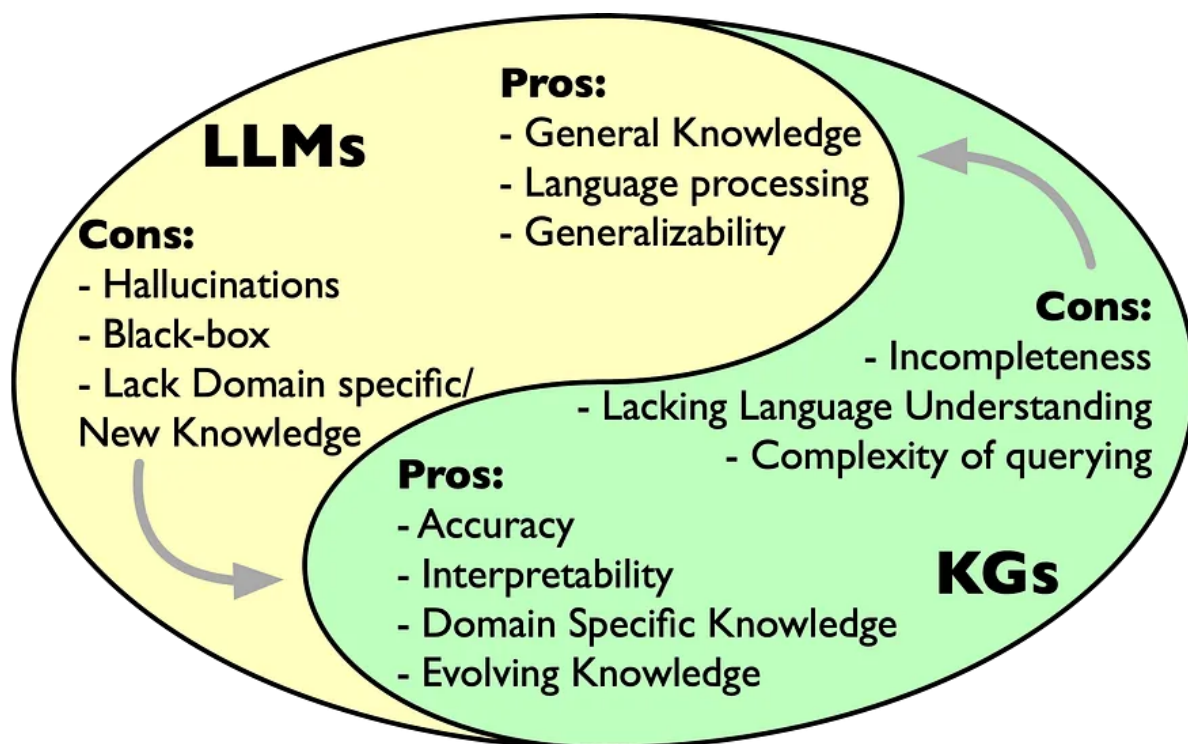
根据帕兰提尔的定义，本体包含**对象、关系、动作、回写、流程、场景**等；它把本体看做是一个组织的数字孪生。可见，帕兰提尔的概念有别于本体的传统定义，具有鲜明的动态特征。

帕兰提尔的本体论实现形式是其产品 Gotham 和 Foundry；传统的本体论实现典型的如知识图谱。我们不妨将这些实现统称为“语义层”。

本文中，“本体论”指理论，“本体”和“语义层”是同义词，指其实现。

知识图谱和GraphRAG

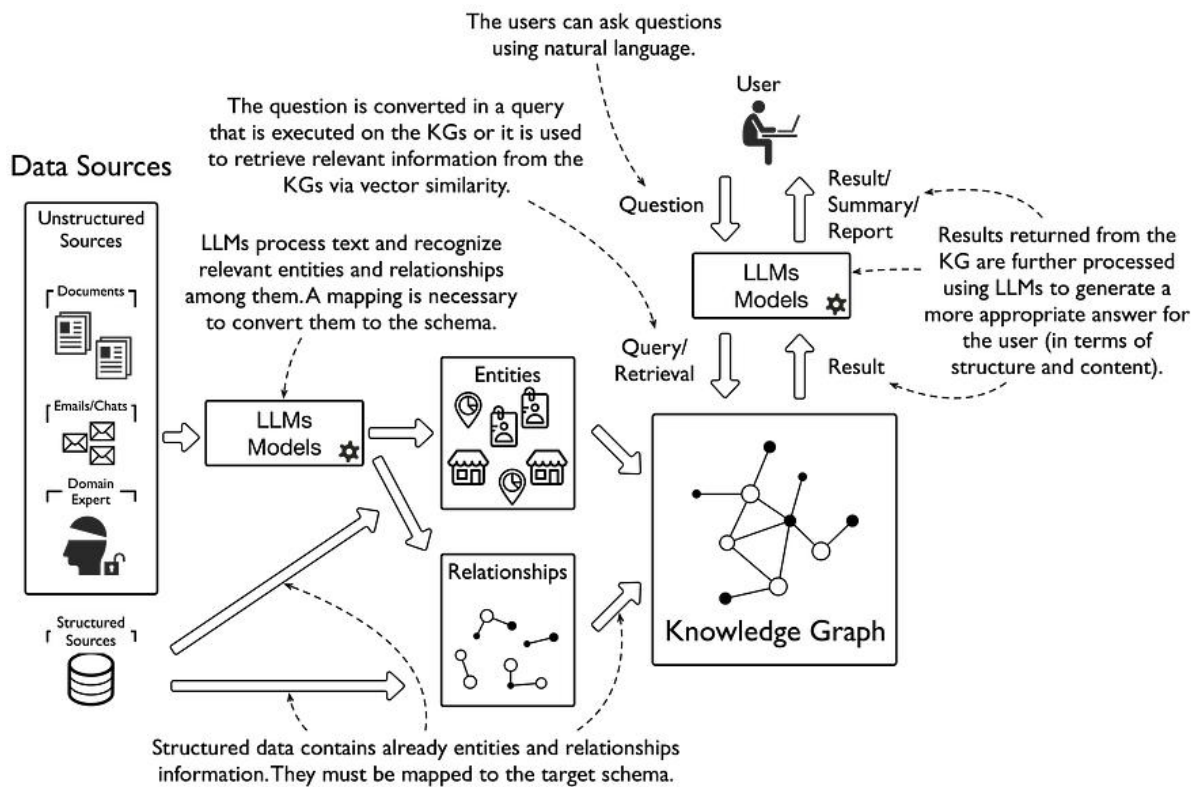
对大多数国内公司而言，知识图谱（Knowledge Graph）类产品是落地语义层的现实选择。下图展示了知识图谱和 LLM 的互补性：



传统上，知识图谱的构建比较费时费力，但在 AI 时代，人们发现可以借助 LLM 来生成知识图谱，微软还开源了 GraphRAG。除了微软的产品，人们也常常使用图数据库如 Neo4j 来实现自己的 GraphRAG 方案，也有轻量级框架如 LightRAG。但无论如何，构建一个高质量的知识图谱都不是轻而易举的，除了传统的挑战如共指消解、实体消歧等，还面临着现代本体对实时性（动态性）的要求。

使用 LLM 建成知识图谱后，就可以基于该图谱进行查询和推理了。这些查询和推理（尤其是多跳推理）的结果，又可以送入 LLM，以帮助实现更精确的问答、统计和根因分析。

下图展示了有知识图谱参与的问答以及利用 LLM 从数据源抽取知识图谱的过程：



由于看到了这个方向的潜力，此类产品正在纷纷出炉；如 Redis 的 FalkorDB、清华的 Tree-KG 等。

帕兰提尔的本体产品为什么不是知识图谱？

从帕兰提尔的 Foundry 看，它并不是知识图谱类产品，为什么呢？

让我们梳理时间线如下：

- 2000年左右的时候，谷歌为了解决大量搜索结果排序的问题，发明了 PageRank 技术。它把网页间的相互引用做成一张图，节点是网页（url），边是引用。被引用得越多，就认为越重要，会优先返回给用户。
- 2012年，谷歌更新了它的搜索引擎，口号是：“Things, Not Strings”，并正式提出“知识图谱”这个概念。它除了返回传统的网页列表外，还返回“知识卡片”（位于搜索结果列表的右侧）。
- 2024年，谷歌又提出 GraphRAG。它的理念是：它是 RAG，但是在 R 阶段，不仅去传统知识库里搜索，还去图数据库里搜索。（在原始论文里，图谱库和向量库是一个，是存在一起的。）
- 2025年，帕兰提尔大火，带火了本体论。但是帕兰提尔的本体论只是在精神上和正统本体论（RDF/语义网）保持一致，实现上则完全不同。帕兰提尔的本体平台更像个对象存储库（或曰“对象中台”）。

因此我们能够回答帕兰提尔的平台为什么是这个样子的。知识图谱是 2012 年才提出的；帕兰提尔成立于 2003 年，在 toC 界，那是万维网的青春期；在 toB 界，则是 OOP 和 J2EE 的青春期。帕兰提尔在哲学上采用了 toC 界的概念，在实现上则受 toB 界的影响更大；这可以理解，因为 RDF 要到 2004 年才出现，而 OOP 思想和 J2EE 应用服务器已日臻成熟。帕兰提尔的产品带有那个时代的典型印记，实际上是以自己的方式实现了 EJB 的愿景。它的“本体层”与其说是语义模型层，不如说是对象模型层（在那个时代也不算错，那时提出“对象”就是为了承载语义，以区别于“数据”）；它对“关系”的定义甚至退化到数据库的一对一/一对多/多对多。它的本体层又分为“语义层”、“动作层”、“动态层”。其中语义层实际上是对象层，而动作层是对象层本来就有的（方法），后来受事件驱动编程的影响，加了事件；动态层则是数据分析、机器学习等技术的应用。可见，它也在不断吸收业界最新的发展成果；LLM 出现后，它搞 AIP，就是在继续吸收这种成果。

语义层的数据源

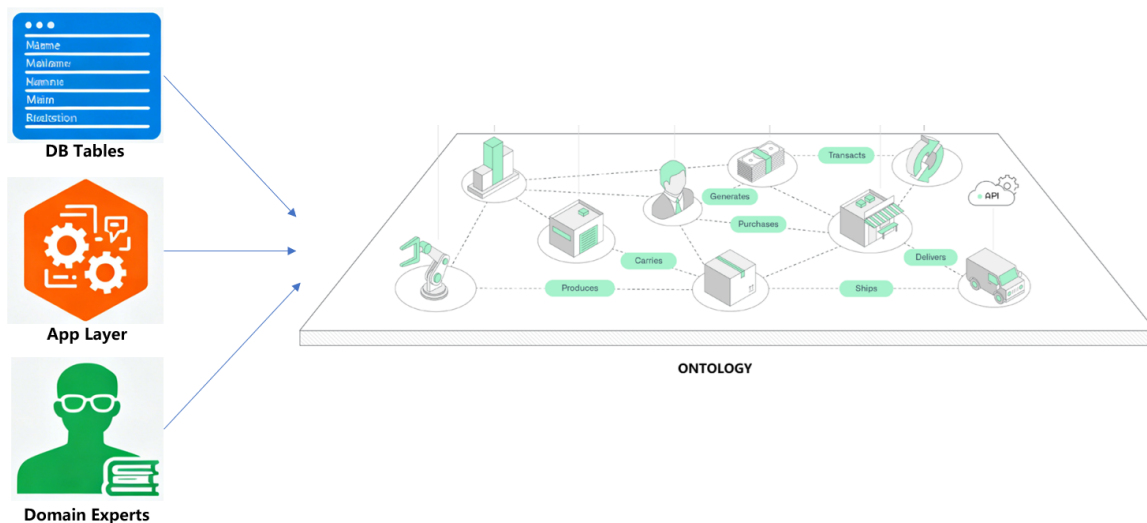
由于我们的着眼点是构建特定 IT/软件系统，不是一般的企业知识库，因此语义层的数据源——构建语义层所依赖的材料——非常重要，它将充当后续业务和 IT 活动的单一事实来源。

对于一个待开发的系统，语义层的数据源应该是业务分析报告和产品需求文档（PRD）。概念模型即抽取自这些文档，或者（最好是）这些文档在撰写的时候就梳理了概念模型。我们可以将这些文档规定为某种“课本”。课本既有形式化的结构（目录、篇章、提要、小结、词汇索引等），又是用自然语言编写的。我们还可以根据实际情况增强它的形式化特征，从而增强机器可读性。在图谱构建这项工作中，课本是人与大模型之间的中介、抓手。毕竟，课本是全球人类学习知识的主要形式；这也是从 Tree-KG、BookRAG 这类项目获得的启发。

对于存量系统，如果要为其补充语义层，有人认为应当从数据层进行逆向工程。逆向工程的思路（Data-First / Bottom-Up）并不算错，但注意不要对“数据”的理解陷于片面。很多人把数据层理解为数据库表，这是不对的。**数据是一个抽象概念，不依赖于媒介**。就现代应用系统而言，数据可以表现为库表中的记录，也可以表现为应用层的表单、服务参数与返回值以及面向领域建模（DDD）中的 VO、DO、DTO 等；此外，业务逻辑、流程、规则也是构建语义层必不可少的“数据”。

关系型数据库 RDBMS 是建立在实体-关系模型理论上的，它对“关系”的刻画能力很弱，只有简单的一对一、一对多、多对多。这是一种纯机械的、面向技术实现的关系，并无业务语义表达。面向对象编程（OOP，同样诞生于上世纪90年代）则在这个基础上向前迈了一步，它的 UML 图至少可以表达引用、组合、聚合等概念；方法则不仅规定了对数据的操作，还能在一定程度上补充（暗示）语义关系。DDD 则进一步强调概念建模的重要性，但它只是“OOP done right”，并没有大的创新。

可见，从数据库表抽取语义层将面临巨大的语义损失；从应用层抽取稍好一点，但仍远远达不到知识图谱所要求的刻画能力。单纯的逆向工程无法构建坚实的语义层，必须辅以有领域专家参与的治理工程。



语义层的更新

今天的人们不会止步于构建某个领域的静态知识图谱，而是会努力**让这个图谱根据实际情况变化**。

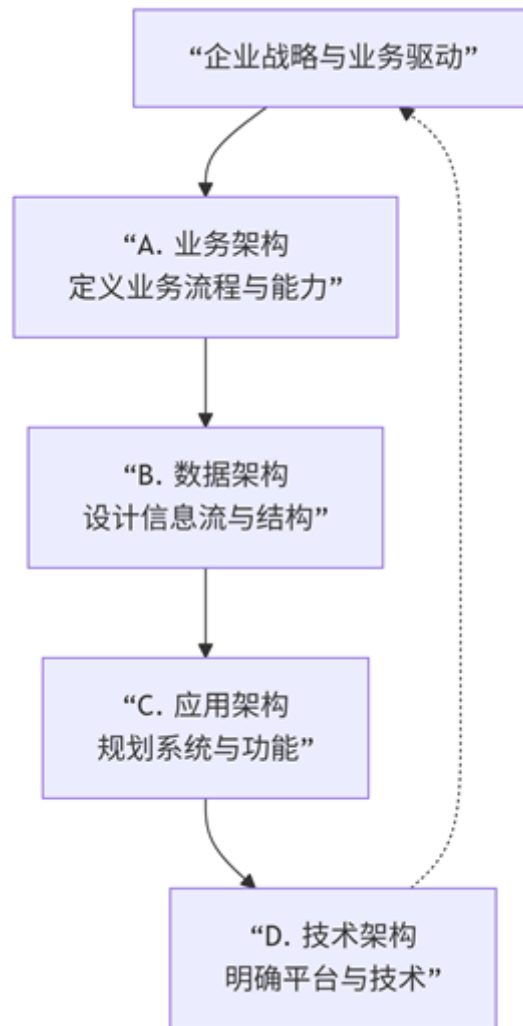
知识图谱的更新有很多种，比如从规模上可分为全量更新和增量更新，从种类上可分为概念更新、实例更新、关系更新等；方法通常是对数据源进行定时抓取或使用 CDC 做实时同步，然后补充或重新生成图谱。

总体而言，知识图谱的更新并不容易，要做到实时就更难了。最近出现的动静孪生的图谱产品，如 Graphiti，或许代表了知识图谱未来的发展方向。

By AI

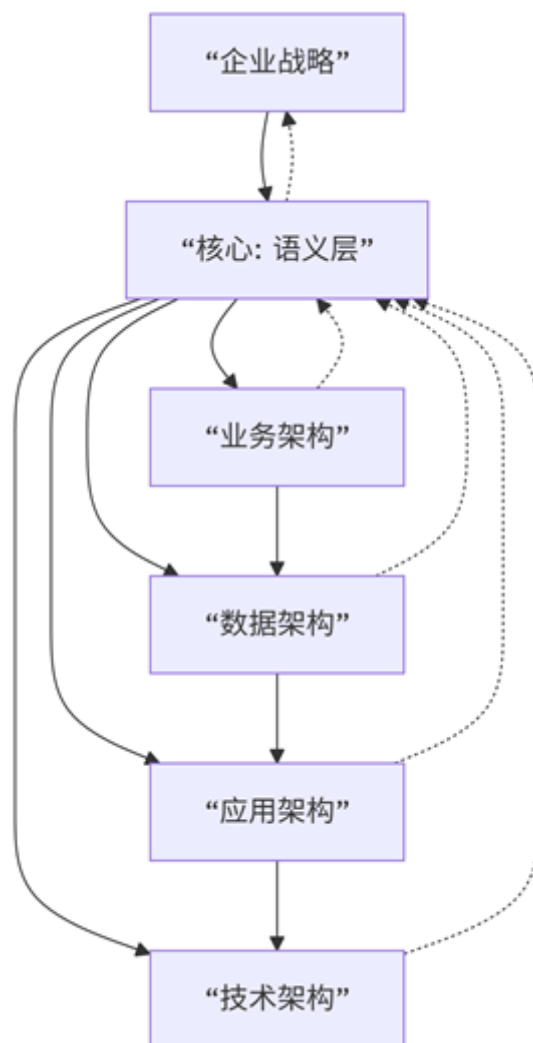
By AI 是指我们的系统是由 AI 帮助创建的，即 AI4SE (AI For Software Engineering) 。

现有的建设企业 IT 软件系统的理论框架仍能发挥重要作用，典型的比如 4A 企业架构。传统的 4A (TOGAF) 实施过程如下图所示：



而从本体论的定义不难看出，语义层的基础是概念模型，在计算机世界中表现为数据模型（这里说对象模型更合适）；因此，**业务架构与数据架构有必要在概念层面上进行融合**。过去，业务架构中也会提信息概念、业务对象，但经常被业务团队忽视；常见的问题有：不够具体、不够形式化、与数据架构断层、侧重流程而非知识等；现在，要让**形式化工作前置**，让企业架构**过度到语义（知识）驱动**，适应 AI 时代，同时也解决业务部门和 IT 部门不使用同一种语言的老大难问题。

这样，**在业务架构阶段，语义层的骨架就站出来统一了概念和规则，从而为后续的数据、应用乃至技术架构提供一致的、机器可理解的指导**。后续各环节则继续丰富语义层的血管和肌肉，最终，这个语义层就是这个系统的领域本体。



可见，语义层并没有改变 4A 架构的实质，只是借 AI 的东风，使其 done right。

业务分析与需求梳理

如前所述，在这个阶段就要构建语义层的骨架，即形式化的概念模型定义和关系定义，后续的业务能力和业务流程梳理则直接使用这些概念和关系。

业务分析师和产品经理应当借助 AI 工具来从业务和需求文档中抽取概念、关系、规则和流程；如果有条件，则应当直接编写这些形式化内容。

这里需再次强调概念模型的完备性。在这个阶段结束时，梳理出来的概念应该已经涵盖了数据架构的主体部分（上面我们提到过业务架构和数据架构的融合）。

系统设计

基于上一个阶段产生的基础语义层，设计人员（架构师、开发经理）应当使用 AI 工具或智能体来辅助进行系统设计；并在此过程中补充、完善语义层的概念和逻辑。

设计是实现的前一环；在本环节应当完成 90% 以上的语义层建设，否则它将难以胜任指导系统实现的重任。

不难看出，在 AI First 的语境里，业务分析师、产品经理和架构师的角色正在进一步压合；**开发工作正在左移，而设计工作正在右移**。未来的组织形态则是自然人驱动角色化智能体。

开发与测试

这是系统实现环节。AI 的一个重要应用场景就是编程。

在大模型时代，编程智能体应成为开发人员的标配，开发团队应尽快过渡到 AI First 的范式上来，并根据自身情况选择语义层和编程智能体的连接方案。

语义层使这些智能体“做对的事情”，规范（Spec）或技能（Skill）则使之“把事情做对”。以下是一些被业内采用规范：

- OpenSpec
- Spec-Kit
- BMAD-METHOD
- Kiro
- RIPER-5

由编程智能体根据规范来进行应用开发的方法叫做“规范驱动开发（Spec Driven Development, SDD）”。

当然，开发与测试阶段仍担负着进一步完善语义层的任务。

监控与运维

运维领域有所谓 AIOps（AI for IT Operations）。

将单个系统的语义层集成进运维体系（智能体）后，就可以做个性化的监控、预测预警、资源优化、根因分析、恢复策略——最终达到应用自治。

For AI

For AI 讨论 AI Native 应用和传统应用如何适配 AI 时代。

AI Native

我们上面所说的 AI 原生应用本身就是 AI Ready 的。系统建设过程中产生的语义层就是它被 AI 读取、理解的关键。同时，AI Ready 还意味着工具层面上的就绪，比如语义层查询接口已就绪，业务 API 已完成工具化（遵循 Tool Calling 或 MCP 规范，可供智能体调用），等等。

至于是否每个应用都需要一个专属的智能体，与其它智能体组成多智能体系统，则可根据实际情况而定。

存量治理

针对企业的存量系统对接 AI 时代的问题，我们在“语义层的数据源”一节略有提及，更多的内容请参考《[A2I：企业AI基础设施建设思路](#)》，此处不再赘述。

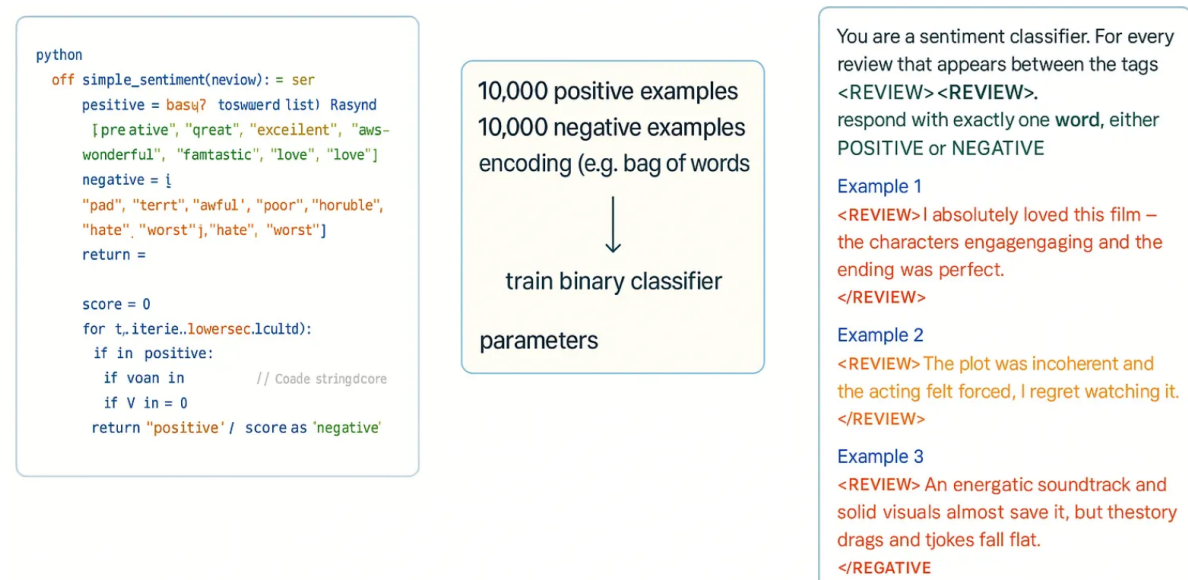
未来展望

应用的未来形态

有人把软件分为三代，如下：

- 软件 1.0：逻辑过程即软件
- 软件 2.0：数据处理即软件
- 软件 3.0：意图执行即软件

第一代软件以各种编程语言为代表，第二代以神经网络技术为代表，第三代则以使用 LLM 的操作系统为代表。示意图如下，下方是它们的编写形式。

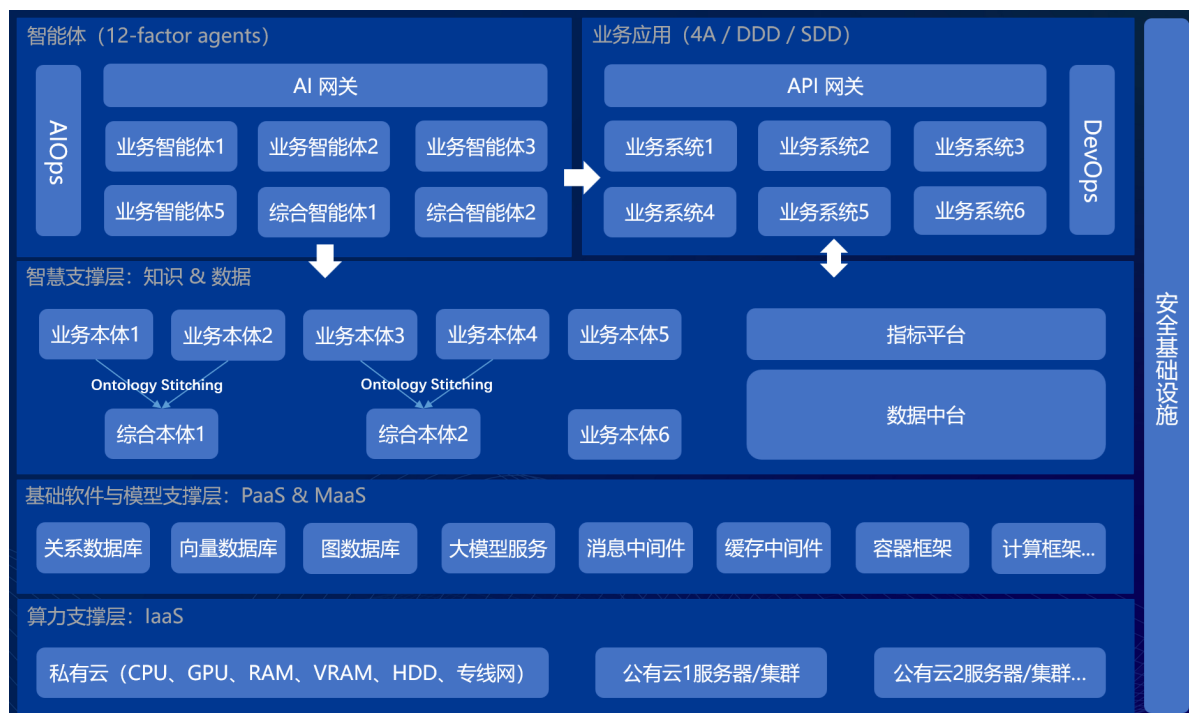


至此，软件开发人员应该意识到一个问题：现有应用（第一代软件）的地位。

许多现有的应用软件，尤其是企业应用，都是在查询和操作数据。之所以需要一个应用层而不是直接操作底层数据库，其实就是为了添加语义，这在 OOP 和 DDD 上表现得再明显不过了。即，**现存应用本身就是上个时代的、机械的、固定流程式的语义层**。如果再联想到从低级编程语言到高级编程语言的跳变，就更能明白，计算机软件发展的一个重要脉络就是不断脱离机器语言、靠近人类语言。

当人工智能和本体发展到足够严谨和精确的程度时，第一代软件就会被废除；当它继续发展到能胜任编写第二代软件时，第三代软件调用第二代软件的执行范式也就同样迎来了黄昏。

但是在那个（遥远的）时代到来之前，既有的软件范式将继续存在，并且与 LLM /智能体长期共存，构成一种过渡形态：



上面的架构图中，我们引入了**智慧支撑层**，它是这种架构形态的核心。这一层除了包含我们说的语义层外，还包含数据中台和指标平台，后者主要解决传统指标/报表过于固定，新增时需要编程开发的问题。

- 首先，我们有6个业务本体（语义层），它们构成右上方6个业务系统的基础；
- 6个业务系统在各自本体的基础上被开发出来（如果是存量系统，则是由领域专家从业务系统抽取出本体）；
- 业务系统1、2、3、5都有对应的智能体（左上方），这些智能体访问智慧支撑层以获取知识和数据，访问业务系统以调用其 API；
- 业务本体1和2被缝合成一个综合本体，3和4被缝合成另一个综合本体，是为了支持跨领域智能体，即综合智能体1和综合智能体2；
- 智慧支撑层与业务应用间的箭头是双向的；因为业务应用要访问语义层以构建自身，而数据中台需访问业务应用以获取数据；
- 业务应用开发的指导框架是 4A，设计方法是 DDD，实现方式是 SDD。智能体开发的指导思想是 12-factor agents，实现上则有很多框架可用。

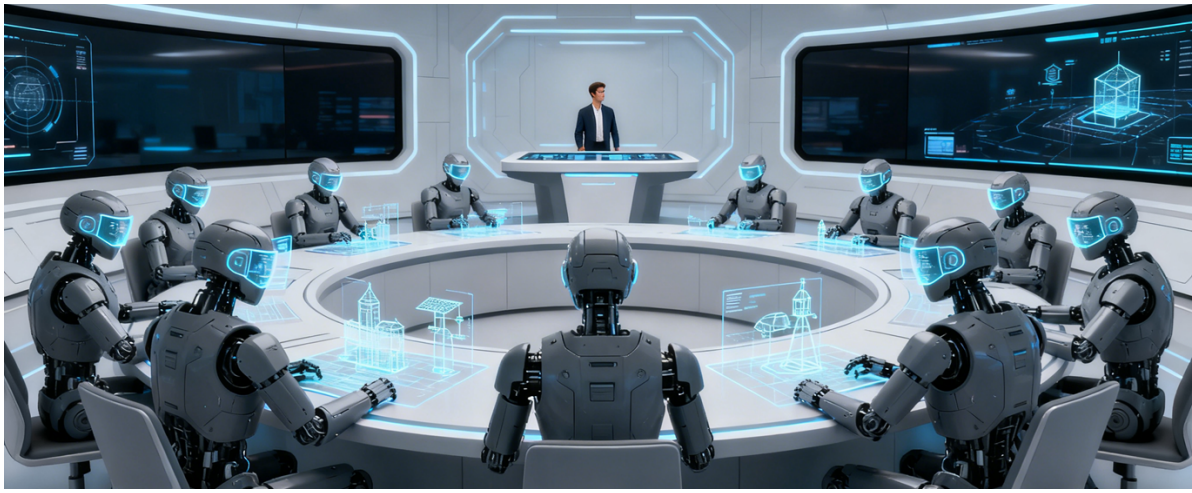
组织的未来形态

与应用的未来形态对应的，是 IT 组织的未来形态。

与其它组织一样，IT 部门也在从事某项业务（领域），即 IT 系统的建设；因此它自身也需要本体。需求文档的格式规范、开发测试的规则规范、运维体系规范等，都是 IT 组织本体的数据源。未来组织需要的，可能只是少数本体建设与运维专家，他们同时也是 IT 领域的专家。

至于非 IT 组织，则只需要领域专家和中等成本的本体运维人员。

这种组织被称为 AI 优先（AI First）型组织。它不是一蹴而就的，通常沿着“人类+AI 助手”→“人机混合团队”→“人类决策、AI 执行”这样的路径发展而来。



参考资料

1. [连接AI与决策：深度解析Palantir的“基石”：本体（Ontology）](#)
2. [Palantir 的“本体工程”的核心思路、技术架构与实践示例](#)
3. [业务语义：从Ontology构建到大模型集成的技术路径探析](#)
4. [基于本体论与大模型的新一代智能应用开发体系](#)
5. [驾驭AI：一位全流程工程师的软件工程智能化实践与思考](#)
6. [为什么需要知识图谱？什么是知识图谱？——KG的前世今生](#)
7. [语义网络，语义网，链接数据和知识图谱](#)
8. [\[经典之作\] 大语言模型与知识图谱的融合：通往智能未来的路线图](#)
9. [命名实体识别、实体消歧、实体统一、指代消解、关系抽取](#)
10. [AI Infra: FalkorDB, 给AI用的图数据库](#)
11. [Graphiti: 快速将知识图谱塞进智能体,为更智能的 AI 代理构建实时知识图谱](#)
12. [当Agent有了“长情”的大脑：深度拆解与对比Mem0/Graphiti/Cognee三大开源Memory方案](#)
13. [上不了生产环境的Agent，都是耍流氓式的自嗨！](#)
14. [微软：人类与智能体协同时代的未来组织蓝图](#)
15. [麦肯锡最新洞察：智能体组织（Agentic Organization）——AI时代全新组织范式，与你我都相关](#)
16. [Three Metaphors for Ontology](#)
17. [Taxonomy, Ontology, Knowledge Graph, and Semantics](#)
18. [Ontology Stitching: How to Align/Merge Enterprise Knowledge Graphs \(A Practical Guide\) and part 2](#)
19. [Why Agentic AI Needs a Semantic Core](#)
20. [Building the brain of your business for actual AI automation](#)
21. [What is a Semantic Layer? \(Components and Enterprise Applications\)](#)
22. [What is a Semantic Architecture and How do I Build One?](#)
23. [Foundry ontology core concepts](#)
24. [Enterprise AI Architecture Series: How to Build a Knowledge Intelligence Architecture \(Part 1\)](#)
25. [Enterprise AI Architecture Series: How to Extract Knowledge from Unstructured Content \(Part 2\)](#)
26. [Enterprise AI Architecture Series: How to Inject Business Context into Structured Data using a Semantic Layer \(Part 3\)](#)
27. [The Semantic Layer: Your AI Agent's New Best Friend \(or Why They Keep Asking About "Revenue"\)](#)
28. [Unifying Large Language Models and Knowledge Graphs: A Roadmap](#)
29. [Retrieval-Augmented Generation with Knowledge Graphs for Customer Service Question Answering](#)
30. [12 Factors Agent - How to build production grade AI-Agents](#)
31. [12-factor-agents](#)

32. [AI Agent Architecture: Tutorial & Examples](#)
33. [Challenges and Paths Towards AI for Software Engineering](#)